

CS620T: Compilers - Principles and Implementation

Autumn 2026

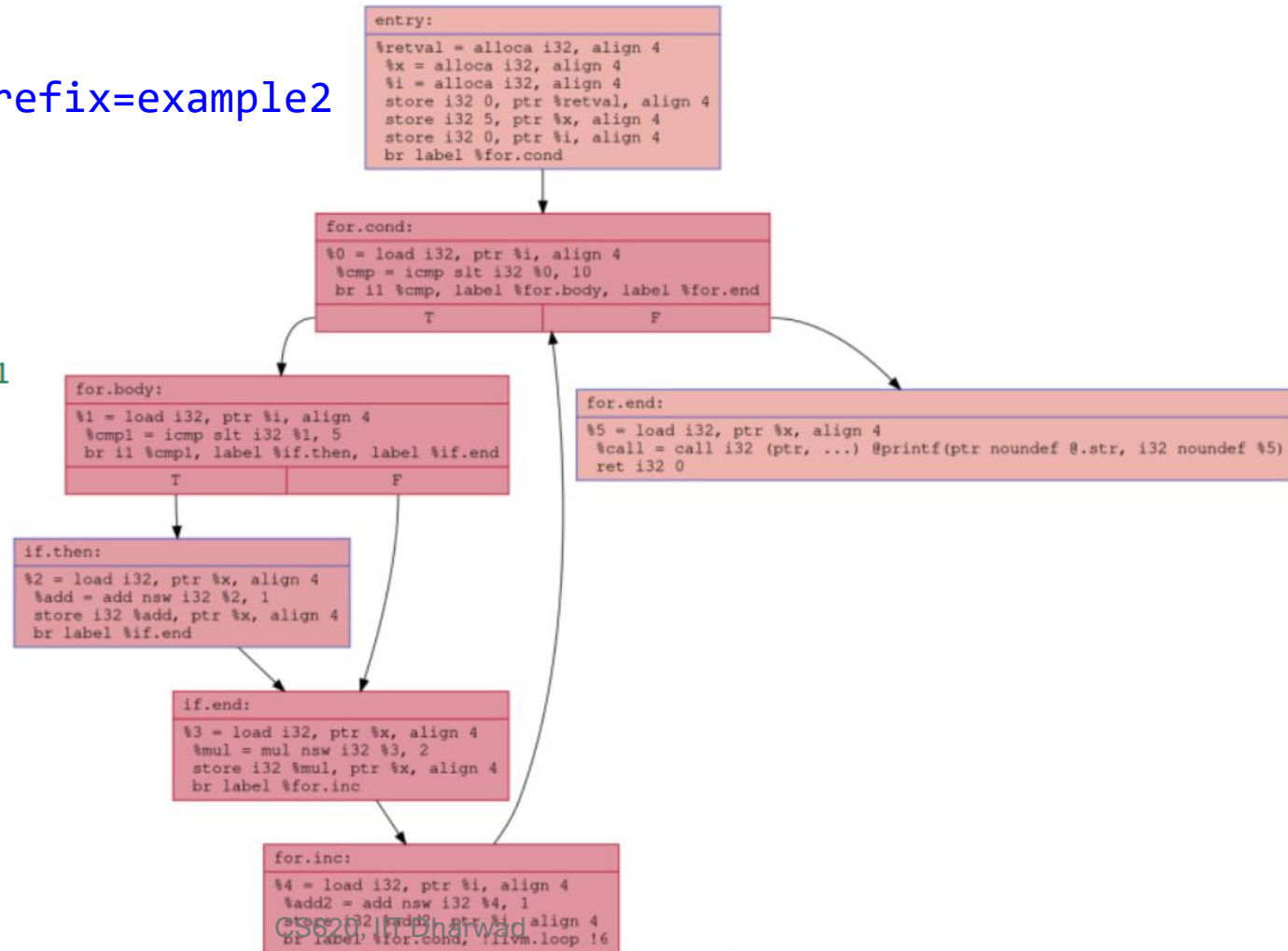
Week10: Demo of concepts learnt last week, Loop Optimizations

Flow Graphs - Representation

opt -passes=dot-cfg
-cfg-dot-filename-prefix=example2
example2.ll

```

3  int main() {
4      int x, i;
5      x = 5;
6      for(i=0;i<10;i=i+1
7          |   if (i < 5)
8              |       x = x+1;
9              x = x*2;
10     }
11     printf("%d",x);
12     return 0;
13 }
    
```

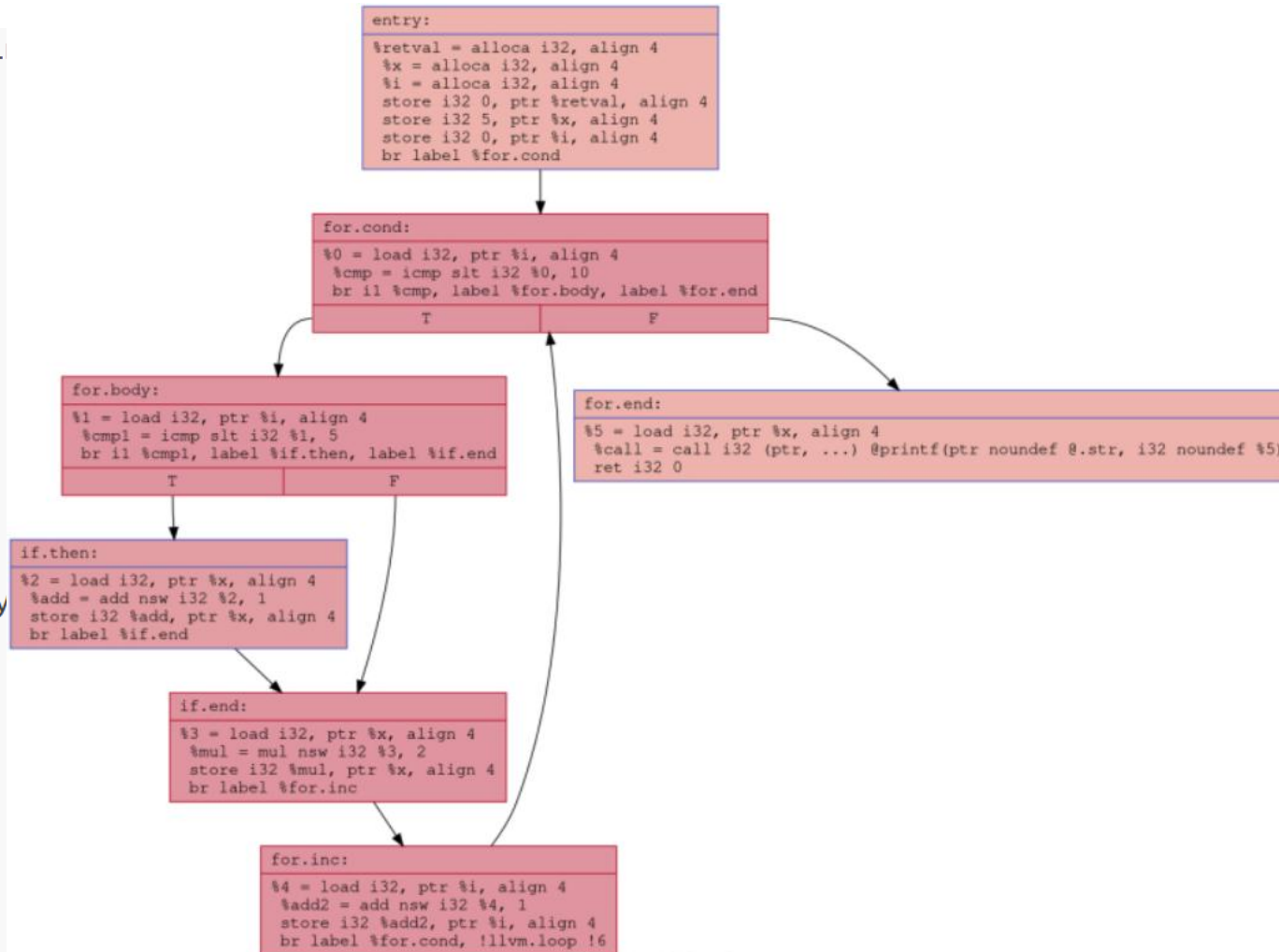


CFG for 'main' function

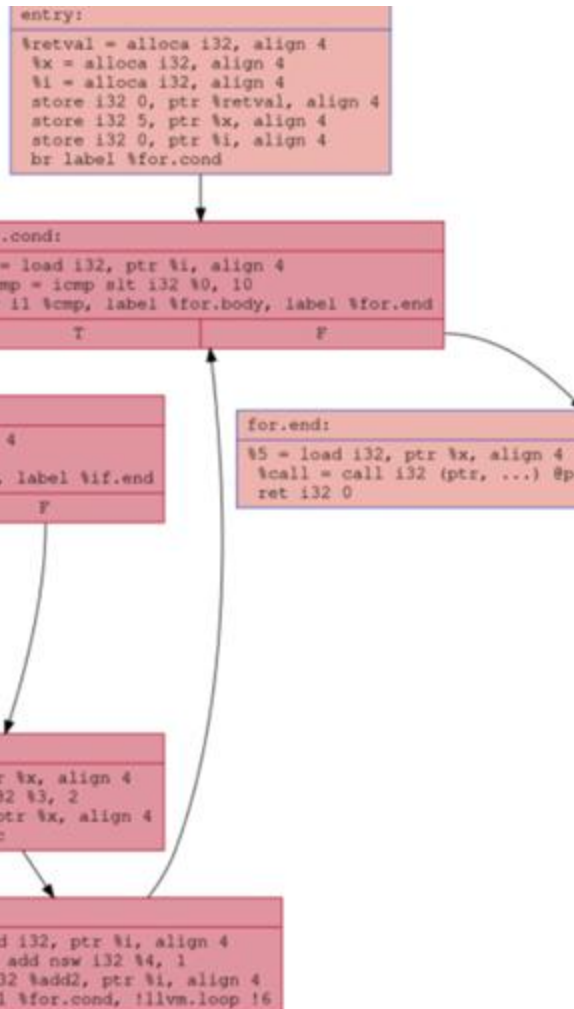
Dominators

opt-load-pass-plugin=./fnmodpass/build/MyPass.so -passes="my-module-pass" -disable-output \$(TESTCASE).bc

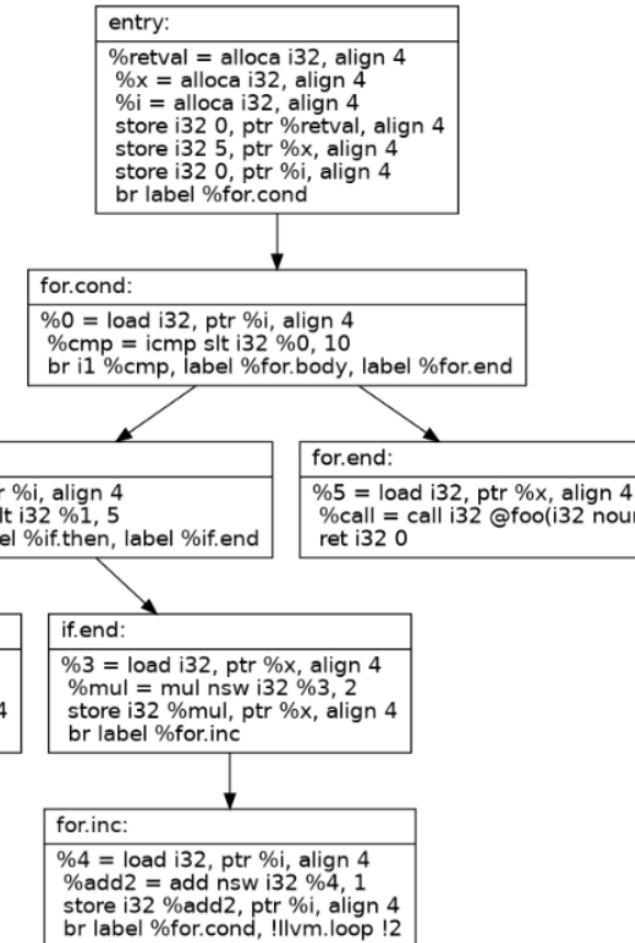
[ModulePass] Function: mai
entry dominates entry
entry dominates for.cond
entry dominates for.body
entry dominates if.then
entry dominates if.end
entry dominates for.inc
entry dominates for.end
for.cond dominates for.cond
for.cond dominates for.body
for.cond dominates if.then
for.cond dominates if.end
for.cond dominates for.inc
for.cond dominates for.end
for.body dominates for.body
for.body dominates if.then
for.body dominates if.end
for.body dominates for.inc
if.then dominates if.then
if.end dominates if.end
if.end dominates for.inc
for.inc dominates for.inc
for.end dominates for.end



Dominator Tree



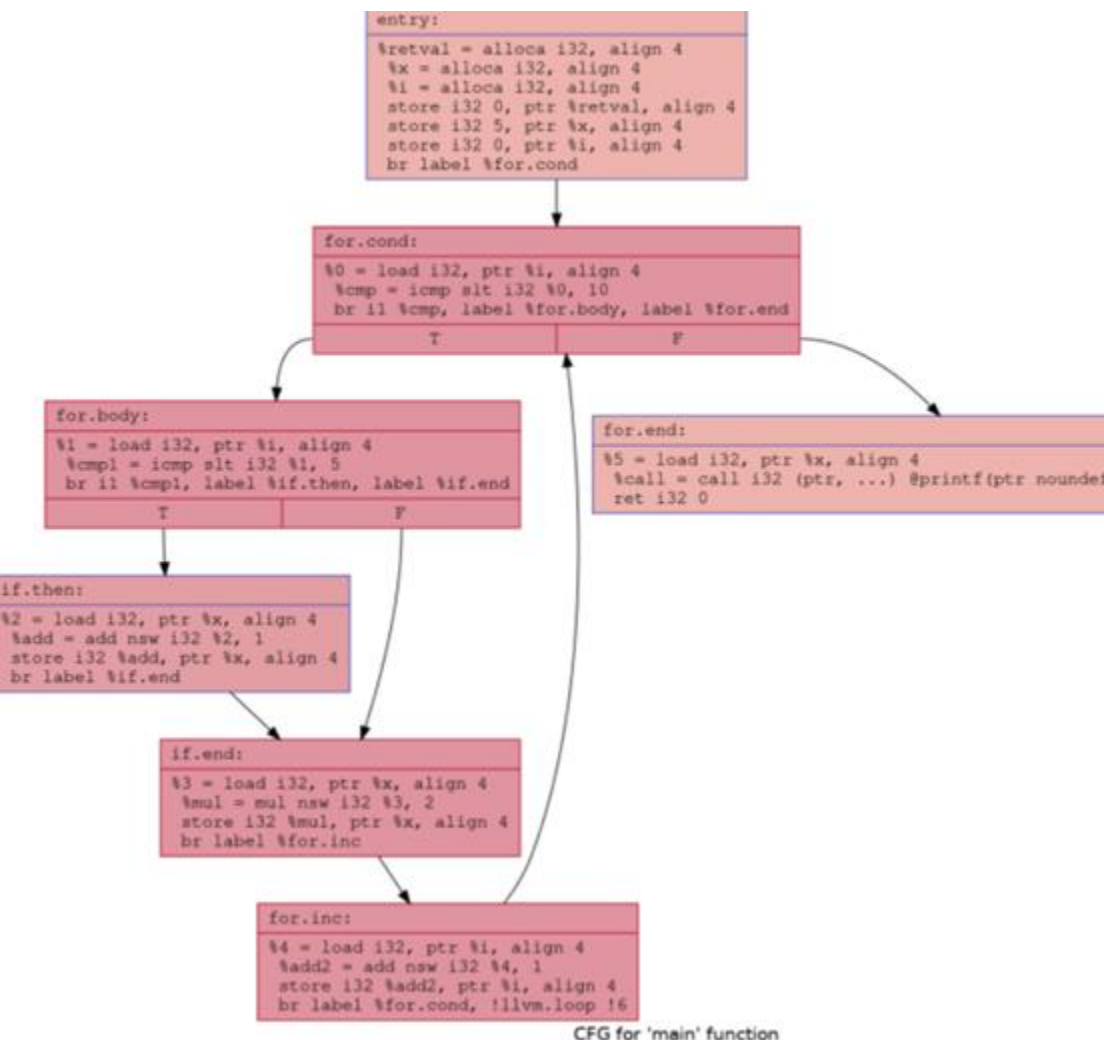
CFG for 'main' function



Dominator tree for 'main' function

opt -passes=dot-dom \$(TESTCASE).ll
 dot -Tpng dom.main.dot -o \$(TESTCASE)_dom.png

Finding Dominance Frontiers



```
[ModulePass] Function: main
Function: main
Dominance frontier of Block entry:
  empty
Dominance frontier of Block for.cond:
  for.cond
Dominance frontier of Block for.body:
  for.cond
Dominance frontier of Block if.then:
  if.end
Dominance frontier of Block if.end:
  for.cond
Dominance frontier of Block for.inc:
  for.cond
Dominance frontier of Block for.end:
  empty
```

Note: `opt -passes=domtree ...`
does not work as domtree is an internal pass

Recall: SSA

- Converting from unrestricted form to SSA form

```
1:      i1 = 1
2: L1:  max = 10
3:      if i1 < max {
4:          i2 = i1 + i1
5:      goto L1;
6:      }
7:      print(i);
```



i1 or i2 ?

Phi nodes – special instructions that help deal with control flow. E.g.,
`i3 = phi [i1, bb1], [i2, bb4]`

- **Where should we insert phi nodes?**

Dominance frontiers

- **What do we need to do after inserting phi nodes?**

Rename variables so that every assignment gets a unique name

Converting to SSA form

- Where do we insert phi nodes?

```
for v in vars {  
  for d in defs[v] {  
    for block in DF[d] {  
      if (block does not have a phi node)  
        add phi node to block  
      if (block is not part of defs[v])  
        add block to defs[v]  
    }  
  }  
}
```

Converting to SSA form

- Rename variables

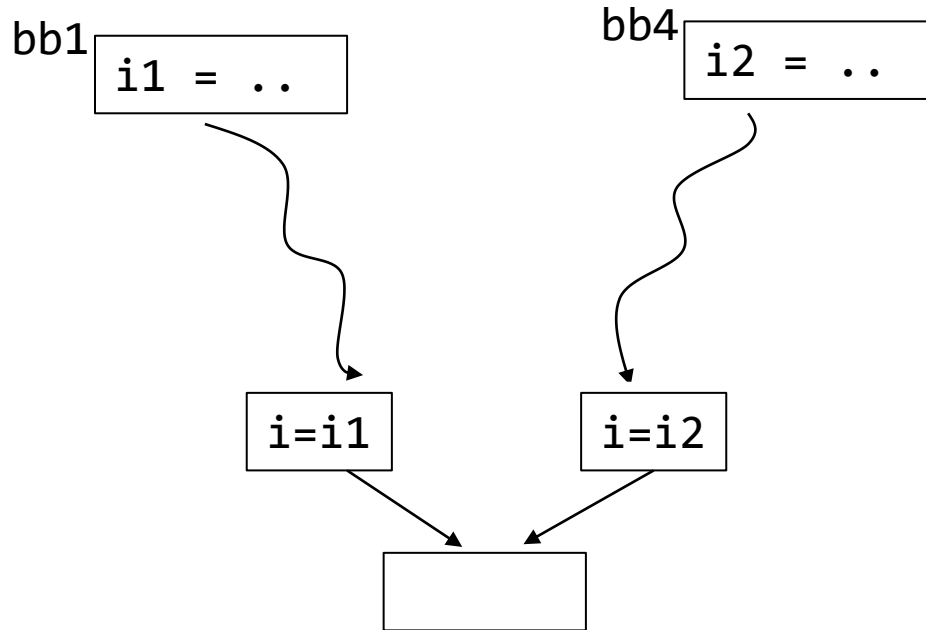
Stack[v] is a stack of variable names for every variable v

```
Rename(block) {  
    foreach instn in block {  
        replace each argument to instn with stack[argument's oldname]  
        replace instn's destination with newname  
        push newname to stack[destination's oldname]  
    }  
  
    foreach s in successor(block){  
        foreach p in s's phi nodes {  
            read stack[v] and plug in p (if p is for v)  
        }  
    }  
  
    foreach b in immediately_dominated_by(block)//need dominance tree here  
        Rename(b)  
  
    pop all names pushed to stack  
}
```

Rename(entry)

Converting **from** SSA form

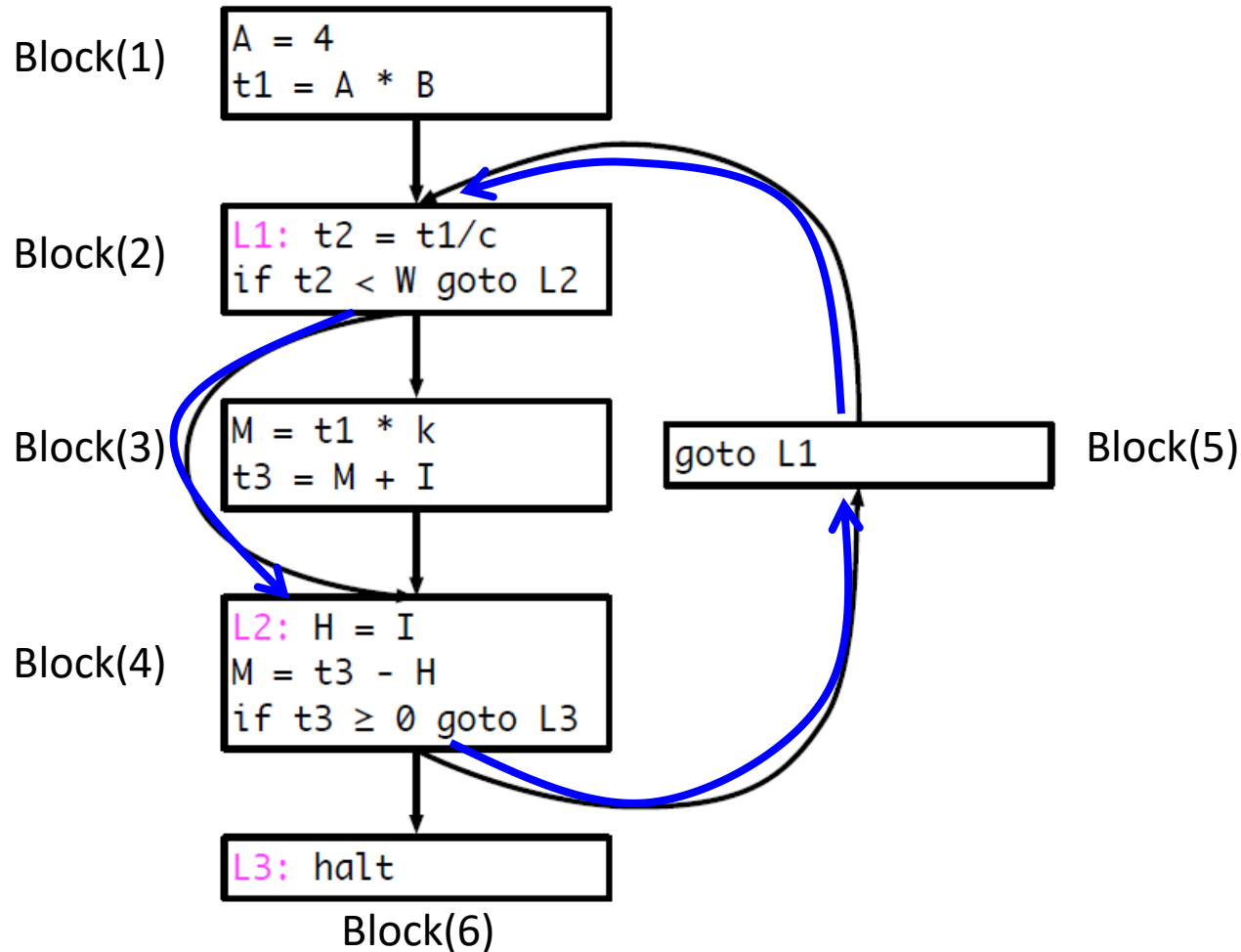
- Get rid of Phi nodes



References

- [Compiler Explorer](#) (browser-based option to view LLVM IR with phi nodes. Use `-O1 -emit-llvm` compiler flag with `phi.c`)
- https://www.llvmpy.org/llvmpy-doc/dev/doc/llvm_concepts.html
- [CS 6120: The Self-Guided Course](#) (lesson 5)

Exercise: Identify Loops in CFGs



Consider: {B2, B4, B5}. Is this a loop? Are there other loops?

Loop Invariant Code Motion

- Optimization that works with natural loops
- Preheader – a unique block that is a predecessor of loop-entry block

Optimize Loops -Identifying Invariant Expressions

- How do we identify expressions that can be moved out of the loop?
 - LoopDef = { } set of variables defined (i.e. whose values are overwritten) in the loop body
 - LoopUse = { } 'relevant' variables used in computing an expression

```
Mark_Invariants(Loop L) {  
    1. Compute LoopDef for L  
    2. Mark as invariant all expressions,  
       whose relevant variables don't belong  
       to LoopDef  
}
```

Optimize Loops -Identifying Invariant Expressions

- Example

LoopDef{}

```
for I = 1 to 100      → {A, J, K, I}
  for J = 1 to 100    → {A, J, K}
    for K = 1 to 100  → {A, K}
      A[I][J][K] = (I*J)*K
```

Optimize Loops -Identifying Invariant Expressions

- Example

LoopUse{}

```
for I = 1 to 100      _____> {}  
  for J = 1 to 100    _____> {I}  
    for K = 1 to 100  _____> {I, J}  
      A[I][J][K] = (I*J)*K
```

Optimize Loops -Identifying Invariant Expressions

- Example

Invariant
Expressions

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100 → { I*J,
      A[I][J][K] = (I*J)*K      Addr(A[i][j]) }
```

For an array access, $A[m] \Rightarrow \text{Addr}(A) + m$

For 3D array above*, $\text{Addr}(A[I][J][K]) =$

$$\text{Addr}(A) + (I*10000) - 10000 + (J*100) - 100 + K - 1$$

*Assuming row-major ordering of storage

Optimize Loops -Identifying Invariant Expressions

- Example

Invariant
Expressions

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100
      A[I][J][K] = (I*J)*K
```

→ { Addr(A[i]) }

For an array access, $A[m] \Rightarrow \text{Addr}(A) + m$

For 3D array above*, $\text{Addr}(A[I][J][K]) =$

$$\text{Addr}(A) + (I*10000) - 10000 + (J*100) - 100 + K - 1$$

*Assuming row-major ordering of storage

Optimize Loops -Factoring Invariant Expressions

- Move the invariant expressions identified

```
Factor_Invariants(Loop L) {  
    Mark_Invariants(L);  
    foreach expression E marked an invariant:  
        1. Create a temporary T  
        2. Replace each occurrence of E in L with T  
        3. Insert T:=E in L's header code  
           //If loop is known to execute at least once,  
           insert T:=E before LOOP:  
}
```

Optimize Loops -Factoring Invariant Expressions

- Example

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100
      A[I][J][K] = (I*J)*K
```

//Invariant Expressions

Optimize Loops -Factoring Invariant Expressions

- Example

```
for I = 1 to 100
  for J = 1 to 100
    temp1=A[I][J]
    temp2=I*J
    for K = 1 to 100
      temp1[K] = temp2*K
```

Optimize Loops -Factoring Invariant Expressions

- Example

```
for I = 1 to 100
  temp3=A[I]
  for J = 1 to 100
    temp1=temp3[J]
    temp2=I*J
    for K = 1 to 100
      temp1[K] = temp2*K
```

Optimize Loops -Factoring Invariant Expressions

- Expressions cannot always be moved out!

Case I: We can move $t = a \text{ op } b$ if the statement dominates all loop exits where t is live

A node $bb1$ dominates node $bb2$ if all paths to $bb2$ must go through $bb1$

```
for (...) {  
    if(*)  
        a = 100  
}  
c=a
```

Cannot move $a=100$ because it does not dominate $c=a$ i.e. there is one path (when if condition is false) $c=a$ can be executed /'reached' without going to $a=100$

Optimize Loops -Factoring Invariant Expressions

- Expressions cannot always be moved out!

Case II: We can move $t = a \text{ op } b$ if there is only one definition of t in the loop

```
for (...) {  
    if(*)  
        a = 100  
    else  
        a = 200  
}
```

Multiple definition of a

Optimize Loops -Factoring Invariant Expressions

- Expressions cannot always be moved out!

Case III: We can move $t = a \text{ op } b$ if t is not defined before the loop, where the definition reaches t 's use after the loop

```
a=5
for (...) {
    a = 4+b
}
c=a
```

Definition of a in $a=5$ reaches $c=a$, which is defined after the loop

Optimize Loops

- Should be careful while doing optimization of loops

```
while J > I loop  
    A(j) := 10/I;  
    j := j + 2;  
end loop;
```

Optimize Loops – Code Motion

- Should be careful while doing optimization of loops

```
while J > I loop  
    A(j) := 10/I;  
    j := j + 2;  
end loop;
```

- Optimization: can move $10/I$ out of loop
- What if $I = 0$?
- What if $I \neq 0$ but loop executes zero times?

Optimization Criteria - Safety and Profitability

- **Safety** - is the code produced after optimization producing same result?
- **Profitability** - is the code produced after optimization running faster or uses less memory or triggers lesser number of page faults etc.

```
while J > I loop  
    A(j) := 10/I;  
    j := j + 2;  
end loop;
```

- E.g. moving I out of the loop introduces exception (when I=0)
- E.g. if the loop is executed zero times, moving $A(j) := 10/I$ out is not profitable

Loop Optimization - Overview

- Low level optimization
 - Moving code around in a single loop
 - Examples: loop invariant code motion, strength reduction, loop unrolling
- High level optimization
 - Restructuring loops, often affects multiple loops
 - Examples: loop fusion, loop interchange, loop tiling

Optimize Loops –Strength Reduction

- Like strength reduction in peephole optimization
 - E.g. replace $a*2$ with $a<<1$
- Applies to uses of **induction variable** in loops
 - **Basic induction variable (I)** – only definition within the loop is of the form $I = I \pm S$, (S is loop invariant)
I usually determines number of iterations
 - **Mutual induction variable (J)** – defined within the loop, its value is linear function of other induction variable, I, such that
$$J = I * C \pm D \quad (C, D \text{ are loop invariants})$$

Optimize Loops –Strength Reduction

```
strength_reduce(Loop L) {  
    Mark_Invariants(L);  
    foreach expression  $E$  of the form  $I * C + D$  where  $I$  is  
    L's loop index and  $C$  and  $D$  are loop invariants  
        1. Create a temporary  $T$   
        2. Replace each occurrence of  $E$  in  $L$  with  $T$   
        3. Insert  $T := I_0 * C + D$ , where  $I_0$  is the initial value of the  
           induction variable, immediately before  $L$   
        4. Insert  $T := T + S * C$ , where  $S$  is the step size, at the end of  
           L's body  
}
```

Optimize Loops –Strength Reduction

- Suppose induction variable I takes on values I_0 , I_0+S , I_0+2S , $I_0+3S \dots$ in iterations 1, 2, 3, 4, and so on...
- Then, in consecutive iterations, Expression $I*C+D$ takes on values

$$I_0 * C + D$$

$$(I_0 + S) * C + D = I_0 * C + S * C + D$$

$$(I_0 + 2S) * C + D = I_0 * C + 2S * C + D$$

- The expression $\ddots$ changes by a constant $S * C$
- Therefore, we have replaced a $*$ and $+$ with a $+$

Optimize Loops – Strength Reduction

- Example (Applying to innermost loop)

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100
      A[I][J][K] = (I*J)*K
    . . .
    temp2=I*J
    temp4=temp2
    for K=1 to 100
      temp1[K]=temp4
      temp4=temp4+temp2
    //S=1
    //C=temp2
```

→

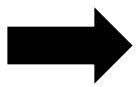
```
for I=1 to 100
  temp3=Addr(A[i])
  for J=1 to 100
    temp1=Addr(temp3(J))
    temp2=I*J
    for K=1 to 100
      temp1[K]=temp2*K
    ↙
```

CS629, MIT OpenWard

Optimize Loops – Strength Reduction

- Exercise (Apply to intermediate loop)

```
for I=1 to 100
  temp3=Addr(A[i])
  for J=1 to 100
    temp1=Addr(temp3(J))
    temp2=I*J
    for K=1 to 100
      temp1[K]=temp2*K
```



```
    . . .
    temp2=I*J
    temp4=temp2
    for K=1 to 100
      temp1[K]=temp4
      temp4=temp4+temp2
```



```
// Induction var = J
// S = 1
// Expression = I * J
```

Optimize Loops – Strength Reduction

- Exercise (Apply to intermediate loop)

...
temp5=I

for J=1 to 100

temp1=Addr(temp3(J))

temp2=temp5

temp4=temp2

for K=1 to 100

temp1[K]=temp4

temp4=temp4+temp2

temp5=temp5+I

... → ...



Optimize Loops – Strength Reduction

- Further strength reduction possible?

```
for I=1 to 100
  temp3=Addr(A[i])
  temp5=I
  for J=1 to 100
    temp1=Addr(temp3(J))
    temp2=temp5
    temp4=temp2
    for K=1 to 100
      temp1[K]=temp4
      temp4=temp4+temp2
    temp5=temp5+I
```

Optimize Loops – Loop Unrolling

- Modifying induction variable in each iteration can be expensive
- Can instead *unroll* loops and perform multiple iterations for each increment of the induction variable
- What are the advantages and disadvantages?

```
for (i = 0; i < N; i++)  
    A[i] = ...
```



Unroll by factor of 4

```
for (i = 0; i < N; i += 4)  
    A[i] = ...  
    A[i+1] = ...  
    A[i+2] = ...  
    A[i+3] = ...
```

Recap: Optimize Loops

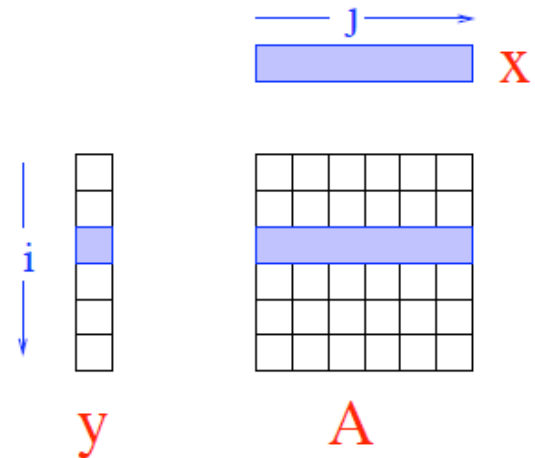
- Low level optimization
 - Moving code around in a single loop
 - Examples: loop invariant code motion, strength reduction, loop unrolling
- High level optimization
 - Restructuring loops, often affects multiple loops
 - Examples: loop fusion, loop interchange, loop tiling

High level loop optimizations

- Many useful compiler optimizations require *restructuring* loops or sets of loops
 - Combining two loops together (*loop fusion*)
 - Switching the order of a nested loop (*loop interchange*)
 - Completely changing the traversal order of a loop (*loop tiling*)
- These sorts of high level loop optimizations usually take place at the AST level (where loop structure is obvious)

Cache behavior

- Most loop transformations target cache performance
 - Attempt to increase *spatial* or *temporal* locality
 - Locality can be exploited when there is reuse of data (for temporal locality) or recent access of nearby data (for spatial locality)
- Loops are a good opportunity for this: many loops iterate through matrices or arrays
- Consider matrix-vector multiply example
 - Multiple traversals of vector: opportunity for spatial and temporal locality
 - Regular access to array: opportunity for spatial locality



$$y = Ax$$

```
for (i = 0; i < N; i++)  
    for (j = 0; j < N; j++)  
        y[i] += A[i][j] * x[j]
```

Loop fusion

```
do i = 1, n
```

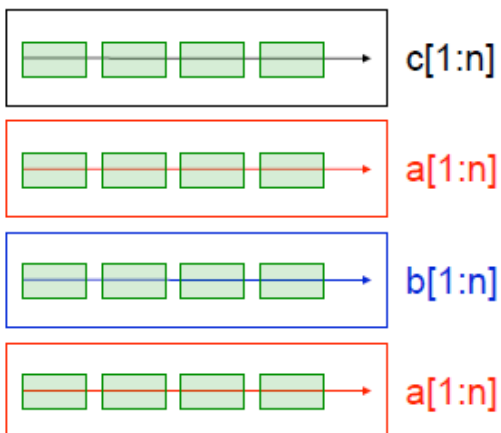
```
  c[i] = a[i]
```

```
end do
```

```
do i = 1, n
```

```
  b[i] = a[i]
```

```
end do
```



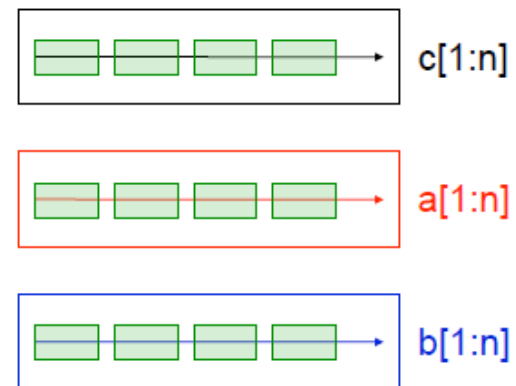
- Combine two loops together into a single loop
- Why is this useful?
- Is this always legal?

```
do i = 1, n
```

```
  c[i] = a[i]
```

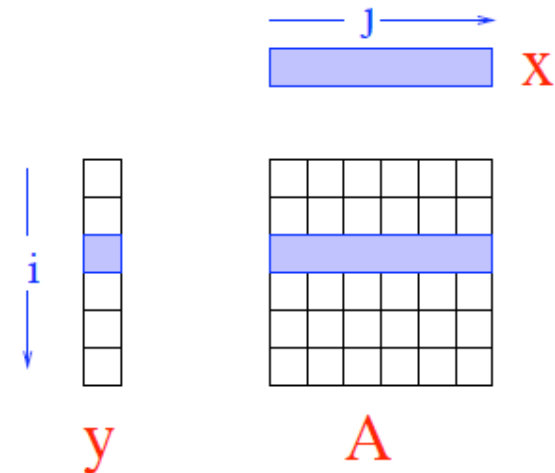
```
  b[i] = a[i]
```

```
end do
```



Loop interchange

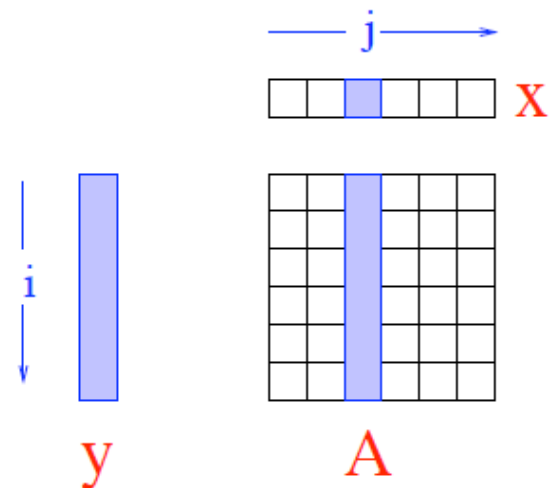
- Change the order of a nested loop
- This is not always legal – it changes the order that elements are accessed!
- Why is this useful?
- Consider matrix-matrix multiply when A is stored in column-major order (i.e., each column is stored in contiguous memory)



```
for (i = 0; i < N; i++)  
  for (j = 0; j < N; j++)  
    y[i] += A[i][j] * x[j]
```

Loop interchange

- Change the order of a nested loop
- This is not always legal – it changes the order that elements are accessed!
- Why is this useful?
- Consider matrix-matrix multiply when A is stored in column-major order (i.e., each column is stored in contiguous memory)



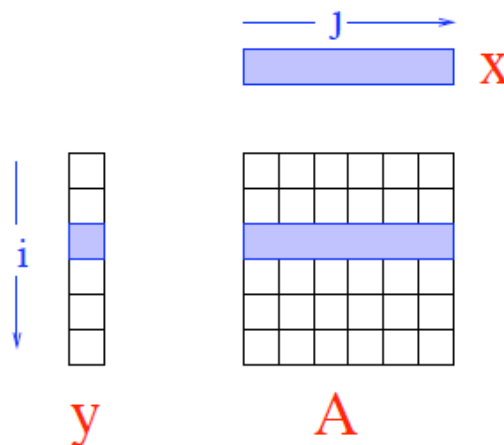
```
for (j = 0; j < N; j++)  
  for (i = 0; i < N; i++)  
    y[i] += A[i][j] * x[j]
```

Loop tiling

- Also called “loop blocking”
- One of the more complex loop transformations
- Goal: break loop up into smaller pieces to get spatial and temporal locality
- Create new inner loops so that data accessed in inner loops fit in cache
- Also changes iteration order, so may not be legal

```
for (i = 0; i < N; i++)  
  for (j = 0; j < N; j++)  
    y[i] += A[i][j] * x[j]
```

```
for (ii = 0; ii < N; ii += B)  
  for (jj = 0; jj < N; jj += B)  
    for (i = ii; i < ii+B; i++)  
      for (j = jj; j < jj+B; j++)  
        y[i] += A[i][j] * x[j]
```

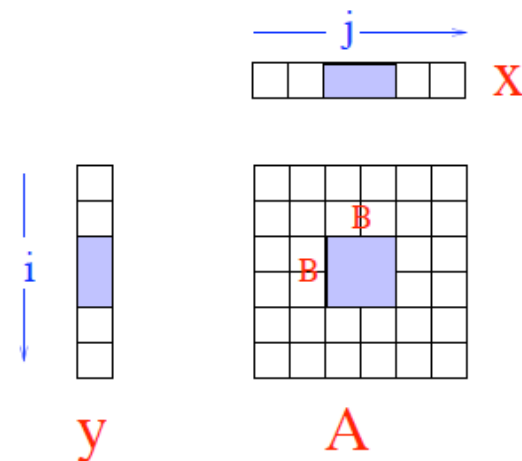


Loop tiling

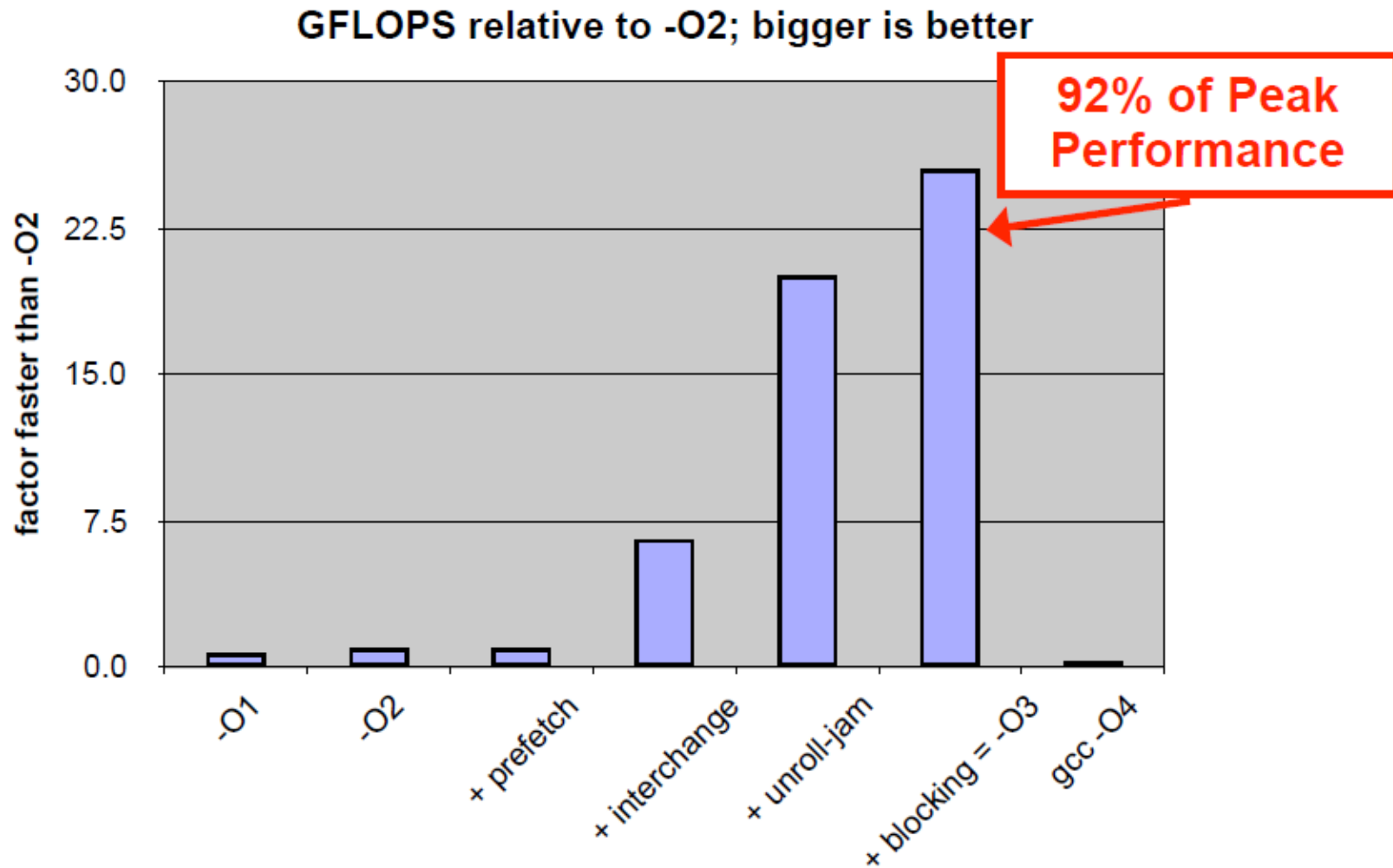
- Also called “loop blocking”
- One of the more complex loop transformations
- Goal: break loop up into smaller pieces to get spatial and temporal locality
- Create new inner loops so that data accessed in inner loops fit in cache
- Also changes iteration order, so may not be legal

```
for (i = 0; i < N; i++)  
  for (j = 0; j < N; j++)  
    y[i] += A[i][j] * x[j]
```

```
for (ii = 0; ii < N; ii += B)  
  for (jj = 0; jj < N; jj += B)  
    for (i = ii; i < ii+B; i++)  
      for (j = jj; j < jj+B; j++)  
        y[i] += A[i][j] * x[j]
```



In a real (Itanium) compiler



Loop transformations

- Loop transformations can have dramatic effects on performance
- Doing this legally and automatically is very difficult!
- Researchers have developed techniques to determine legality of loop transformations and automatically transform the loop
- Techniques like *unimodular transform framework* and *polyhedral framework*
- These approaches will get covered in more detail in advanced compilers course

Dependence Analysis

(Memory Dependence Analysis / Loop Dependence Analysis)